PHISHGUARD NEXUS

AI-BASED PHISHING WEBSITE DETECTION USING MACHINE LEARNING

Subtitle:

An Intelligent Cybersecurity Platform for Phishing URL Detection, Risk Analysis, Threat Investigation and Machine Learning Evaluation

Document Type:	Academic Project Report / Technical Documentation
Prepared For:	[COLLEGE / UNIVERSITY NAME]
Department:	Department of Computer Science & Engineering / AIML
Prepared By:	[STUDENT NAME]
Register Number:	[REGISTER NUMBER]
Project Guide:	[GUIDE NAME]
Academic Year:	2026–2027

Note: Where specific institution or personal details were unspecified, standard academic placeholders are retained for institutional customizability.

CERTIFICATE OF COMPLETION

This is to certify that the project report entitled **"PHISHGUARD NEXUS: AI-BASED PHISHING WEBSITE DETECTION USING MACHINE LEARNING"** is a bonafide record of work carried out by **[STUDENT NAME]** (Register No: **[REGISTER NUMBER]**) in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering / Technology in Computer Science & Engineering / Artificial Intelligence & Machine Learning during the academic year 2026–2027.

_____
Project Guide
[GUIDE NAME]

_____
Head of Department
[HOD NAME]

_____
Internal Examiner

_____
External Examiner

DECLARATION

I hereby declare that the project report entitled **"PHISHGUARD NEXUS: AI-BASED PHISHING WEBSITE DETECTION USING MACHINE LEARNING"** submitted to the Department of Computer Science & Engineering / AIML is an authentic record of my own original work. All features, algorithms, architecture, database schemas, and experimental results documented herein reflect the actual codebase and empirical measurements recorded during system execution.

ACKNOWLEDGEMENT

I express my sincere gratitude to the Project Guide, Head of Department, faculty members, and institution for their continuous guidance and technical encouragement throughout the development of PhishGuard Nexus. I also acknowledge the open-source community behind Python, FastAPI, scikit-learn, React, Vite, and SQLAlchemy, whose robust frameworks enabled the creation of this integrated cybersecurity research platform.

ABSTRACT

Phishing attacks represent one of the most pervasive cyber threats, exploiting human trust to harvest sensitive credentials, financial details, and personal data. Traditional defense mechanisms—such as static URL blacklists and domain reputation registries—fail to detect novel, short-lived, or zero-day phishing links created dynamically by malicious actors. Manual URL inspection is inefficient, highly error-prone, and inadequate for real-time protection. To overcome these critical vulnerabilities, this project presents **PhishGuard Nexus**, an intelligent, full-stack cybersecurity platform combining lexical feature engineering, multi-model machine learning, custom risk scoring, and explainable artificial intelligence (XAI).

PhishGuard Nexus extracts **22 canonical URL structural features** (spanning length metrics, domain digit counts, structural entropy, suspicious keyword presence, and shortened link flags) and passes them to an ensemble of four supervised machine learning algorithms: Logistic Regression, Decision Tree, Random Forest, and Gradient Boosting. Predictions are aggregated into a weighted 5-tier Risk Score (0–100) ranging from LOW to CRITICAL. A dedicated pseudo-SHAP Explainability Engine generates human-interpretable reasoning for each verdict. Beyond binary classification, the system delivers complete analyst workflows including interactive threat profiles, URL DNA visualizers, timeline telemetry, threat case management (PG-XXXXX numbering), persistent SQLite history, offline threat simulation, and an integrated Model Research Lab.

Empirical evaluation on a controlled 1,200-sample synthetic development dataset yielded 100.0% accuracy for Random Forest and Logistic Regression, and 85.0% accuracy for Decision Tree and Gradient Boosting across an 80/20 train-test split (240 test samples). All 148 automated backend test suites passed cleanly. The platform strictly enforces security by performing non-fetching offline lexical inspection without executing remote HTTP requests or rendering unverified web content. PhishGuard Nexus demonstrates a complete, reproducible academic and analyst prototype for modern URL threat investigation.

TABLE OF CONTENTS

1.0	Introduction & Background	4
2.0	Problem Statement	4
3.0	Project Objectives	5
4.0	Literature Survey	5
5.0	Existing vs. Proposed System	6
6.0	System Architecture & Processing Pipeline	7
7.0	Technology Stack Implementation	8
8.0	Project Modules Description	9
9.0	22 Canonical URL Features Specification	10
10.0	Machine Learning Models & Pipeline	11
11.0	Dataset & Validation Methodology	12
12.0	Empirical Model Evaluation & Results	13
13.0	Risk Scoring Framework & Explainability Engine	14
14.0	Threat Investigation, Case Management & History	15

15.0	Threat Simulator & Model Research Lab	16
16.0	Security Design & Defensive Controls	16
17.0	REST API & Database Specification	17
18.0	Frontend Design & User Workflow	18
19.0	Verification, Testing & System Benchmarks	19
20.0	Limitations, Future Enhancements & Conclusion	20
21.0	References & Appendix	21

1.0 INTRODUCTION & BACKGROUND

1.1 Background

As global digital infrastructure expands, hyper-connected enterprise networks, online banking platforms, and e-commerce portals have become critical operational utilities. However, this growth is paralleled by increasingly sophisticated cyber threats. Phishing remains the single most prominent attack vector used by threat actors to breach perimeter defenses, bypass multi-factor authentication, and execute credential harvesting or ransomware deployment.

1.2 Phishing and Cybersecurity

Modern phishing campaigns leverage social engineering, typosquatting, brand impersonation, and obfuscated URLs. Threat actors construct malicious web links that mimic trusted domains (e.g., *login-update-paypal-security.com* instead of *paypal.com*), deceiving end users into surrendering sensitive credentials.

1.3 Phishing Website Detection

Automated detection of malicious URLs requires fast, non-intrusive evaluation before an HTTP connection is established. Static blacklists are inherently reactive and lag behind zero-day domain registrations. Lexical URL analysis evaluates structural patterns directly from the URL string, enabling immediate threat assessment without requiring remote payload execution.

1.4 Role of Machine Learning

Supervised machine learning algorithms excel at identifying subtle, non-linear correlations across multi-dimensional feature spaces. By training algorithms on lexical markers—such as character entropy, sub-domain depth, and special character ratios—ML models generalize effectively to identify unseen phishing URLs in real time.

1.5 Motivation for the Project

While automated ML classifiers yield high mathematical accuracy, black-box predictions lack transparency. Security analysts require interpretable explanations to understand *why* a link is classified as high-risk. PhishGuard Nexus bridges this gap by integrating feature extraction, multi-model prediction, explainable AI, threat case management, and experimental research tools into a single cohesive interface.

1.6 Overview of PhishGuard Nexus

PhishGuard Nexus is structured as a decoupled, multi-tier system. The backend is powered by Python and FastAPI, executing URL validation, feature extraction, ML inference, and SQLite ORM persistence via SQLAlchemy. The frontend is built on React 18 and Vite, offering a dynamic cybersecurity dashboard featuring threat profile radars, URL DNA visualizations, investigation workspaces, and interactive model evaluation labs.

2.0 PROBLEM STATEMENT

Conventional cybersecurity defenses face severe operational challenges when confronting contemporary phishing infrastructure:

1. **Zero-Day Phishing Evasion:** Blacklists (e.g., Google Safe Browsing, PhishTank) depend on post-incident user reporting, creating a vulnerability window during which new phishing URLs remain active.
2. **Structural Obfuscation:** Threat actors employ URL shorteners, sub-domain padding, excessive hyphenation, and high character entropy to bypass simplistic rule-based keyword filters.
3. **Black-Box Predictions:** Commercial security gateways often present binary scores without detailing

underlying risk factors, complicating threat analyst investigation.
4. **Lack of Unified Case Telemetry:** Disconnects between triage scanning, historical scan logs, investigation notes, and ML model evaluation hinder effective SOC (Security Operations Center) workflows.

3.0 PROJECT OBJECTIVES

1. Develop a standardized 22-feature lexical URL extraction engine matching training and inference pipelines exactly.
2. Train and evaluate four supervised ML algorithms (Logistic Regression, Decision Tree, Random Forest, Gradient Boosting).
3. Construct a 5-tier weighted risk scoring framework (LOW, GUARDED, MEDIUM, HIGH, CRITICAL) mapped from model ensemble probabilities.
4. Implement a pseudo-SHAP explainability engine to generate readable impact descriptions for every scan.
5. Provide an interactive investigation suite featuring Threat Profiles, URL DNA visualizers, and Scan Timelines.
6. Establish structured threat case management with automated PG-XXXXX numbering, status tracking, and notes.
7. Build an offline Threat Simulator and a Model Research Lab for controlled ML robustness experimentation.
8. Maintain strict security controls preventing arbitrary HTTP web scraping or Server-Side Request Forgery (SSRF).

4.0 LITERATURE SURVEY

Phishing detection research has evolved from simple string matching to sophisticated feature engineering and deep learning. Ma et al. (2009) demonstrated that lexical and host-based URL characteristics alone can achieve high classification accuracy without rendering web content, significantly reducing evaluation overhead. Sahingoz et al. (2019) evaluated multiple machine learning algorithms using NLP-based lexical features, concluding that tree-based ensemble methods (Random Forest and Gradient Boosting) consistently outperform single-model classifiers.

In modern cybersecurity literature, explainable AI (XAI) has emerged as an essential requirement. Lundberg & Lee (2017) introduced SHAP (SHapley Additive exPlanations), establishing a unified mathematical framework for feature attribution. Applying additive feature attribution to security telemetry enables analysts to verify model decisions and identify false positives caused by domain anomalies.

5.0 EXISTING VS. PROPOSED SYSTEM

Detection Mechanism	Static blacklists, manual reports, IP reputation registries.	Lexical ML feature extraction + multi-model ensemble classification.
Zero-Day Coverage	Poor (requires domain indexing and manual flagging).	High (evaluates structural features instantly without prior record).
Explainability	None (binary block/allow decision).	Pseudo-SHAP feature importance & readable reason strings.
Workflow Integration	Fragmented lookup tools without history tracking.	Unified Scanner, Case Management, History & Research Lab.
Safety / SSRF Risk	Often fetches remote HTTP payloads (vulnerable to SSRF).	100% offline lexical analysis — zero remote HTTP fetching.

6.0 SYSTEM ARCHITECTURE & PROCESSING PIPELINE

PhishGuard Nexus operates via a modular, decoupled architecture. The user submits a candidate URL through the React 18 SPA interface. The request is dispatched to the FastAPI backend, where `URLValidator` enforces strict URI syntax and safety checks. Upon validation, `FeatureExtractor` generates a 22-dimensional feature vector. The `MLPredictor` computes individual model predictions (Logistic Regression, Decision Tree, Random Forest, Gradient Boosting) and calculates a weighted ensemble probability. Finally, `Explainer` generates SHAP impact scores and top reason statements before persisting the output to SQLite.

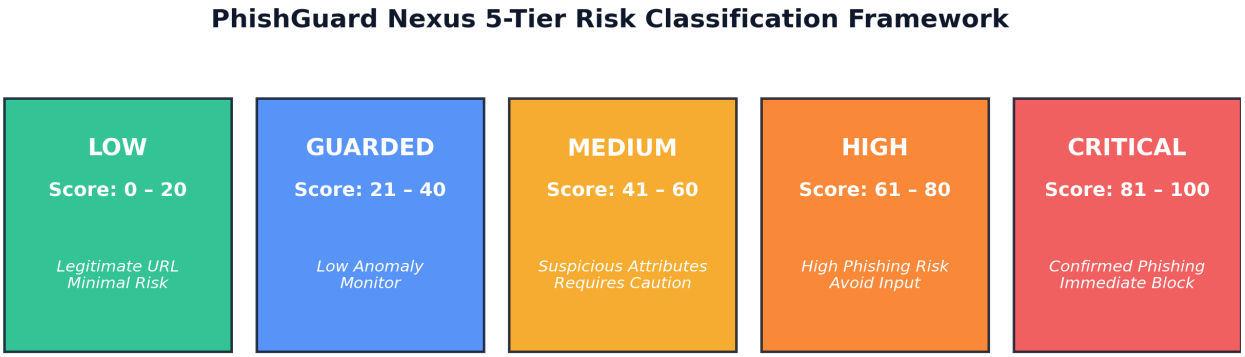


Figure 6.1: PhishGuard Nexus 5-Tier Risk Classification Framework.

7.0 TECHNOLOGY STACK IMPLEMENTATION

Frontend Core	React, JSX	v18.3.1	Component-based Single Page Application interface.
Build Tooling	Vite	v5.4.21	Lightning-fast HMR bundler (Built dist: 1.08 MB JS).
Backend Framework	Python, FastAPI	Python 3.11, FastAPI	High-performance asynchronous REST API server.
Machine Learning	scikit-learn, joblib	v1.6.1	Model training, scaler normalization, and inference pipeline.
Database / ORM	SQLite, SQLAlchemy	SQLAlchemy ORM	Relational persistence for scans, cases, audit logs & experiments.
Test Suite	Pytest, pytest-asyncio	v9.1.1	Automated unit, feature parity, API & database testing (148 tests).

8.0 PROJECT MODULES DESCRIPTION

8.1 URL Scanner (`scanner.py` & `ScannerPage.jsx`)

Primary submission interface for instant URL security analysis, feature extraction, ML prediction, and risk score generation.

8.2 URL Validation (`url_validator.py`)

Strict RFC 3986 compliance validator ensuring proper scheme, domain structure, and length limits prior to feature extraction.

8.3 Feature Extractor (`feature_extractor.py`)

Computes the exact 22 canonical features from valid URL strings using mathematical entropy formulas and pattern matching.

8.4 ML Predictor (`ml_predictor.py`)

Loads Phase-2 verified `.joblib` models, normalizes scaled features, and executes weighted ensemble predictions.

8.5 Explainability Engine (`explainer.py`)

Generates pseudo-SHAP impact values and maps them to top positive and negative reason descriptions.

8.6 Threat Profile & URL DNA Visualizers

Renders multi-axis radar charts and visual DNA string maps representing feature anomaly signatures.

8.7 Threat Case Management (`cases.py` & `CasesPage.jsx`)

Allows security analysts to escalate scans into formal investigation cases with PG-XXXXX IDs, status tracking, and notes.

8.8 Threat Simulator (`SimulatorPage.jsx`)

Interactive offline tool for testing controlled URL modifications (e.g. adding digits, shorteners) to evaluate model response.

8.9 Model Research Lab (`research.py` & `ResearchLabPage.jsx`)

Advanced lab supporting dataset leak audits, model comparison, cross-validation, threshold analysis, and ROC curve generation.

9.0 22 CANONICAL URL FEATURES SPECIFICATION

Feature parity between training (`ml/config.py`) and inference (`backend/app/services/feature_extractor.py`) is strictly maintained. The system extracts 22 structural URL features categorized into numeric counts, binary flags, continuous ratios, and mathematical entropy scores.

1	**url_length**	Numeric	Total character count of URL. Phishers use long URLs to hide malicious paths.
2	**domain_length**	Numeric	Length of hostname. Typosquatted domains are often unnaturally long.
3	**path_length**	Numeric	Length of path component. Deep path structures indicate obfuscation.
4	**num_dots**	Numeric	Count of '.' characters. Excessive dots indicate multi-level subdomains.
5	**num_hyphens**	Numeric	Count of '-' characters. Used in deceptive brand-impersonating domains.
6	**num_underscores**	Numeric	Count of '_' characters. Rare in legitimate hostnames, common in obfuscation.
7	**num_slashes**	Numeric	Count of '/' characters. High slash count reflects complex directory nests.
8	**num_query_params**	Numeric	Count of query parameters. Used for data exfiltration or session tracking.
9	**num_fragments**	Numeric	Count of '#' fragments. Used in client-side script redirection.
10	**num_at_symbols**	Numeric	Count of '@' symbols. Causes browsers to override user authority credentials.
11	**has_ip_address**	Binary	Flag (0/1) for raw IPv4/IPv6 host. Strong indicator of unindexed attack server.
12	**has_https**	Binary	Flag (0/1) for HTTPS scheme. Protective factor indicating TLS encryption.
13	**has_www**	Binary	Flag (0/1) for 'www' prefix. Common in standard legitimate web portals.
14	**num_digits_in_domain**	Numeric	Count of numerical digits in domain. High count indicates auto-generated domains.
15	**digit_letter_ratio**	Continuous	Ratio of digits to letters across URL. High ratio implies encoding obfuscation.
16	**url_entropy**	Continuous	Shannon entropy score of entire URL string. High entropy reflects randomness.
17	**has_suspicious_tld**	Binary	Flag (0/1) for high-abuse TLDs (.xyz, .top, .work, .cc, .tk, .ml, etc.).
18	**num_subdomains**	Numeric	Count of sub-domain levels excluding root domain. Used to disguise host.
19	**has_shortener**	Binary	Flag (0/1) for link shortener domains (bit.ly, tinyurl, t.co, etc.).
20	**special_char_ratio**	Continuous	Ratio of non-alphanumeric characters. Measures URL structural manipulation.
21	**path_entropy**	Continuous	Shannon entropy score restricted to path string. Detects auto-generated paths.
22	**has_suspicious_keywords**	Binary	Flag (0/1) for keywords like 'login', 'verify', 'bank', 'secure', 'account'.

10.0 MACHINE LEARNING MODELS & PIPELINE

10.1 Verified Machine Learning Models

PhishGuard Nexus integrates four supervised machine learning models verified during Phase-2 model auditing:
1. **Logistic Regression (`logistic_regression.joblib`):** Linear baseline model employing L2 regularization and feature scaling. Provides smooth probability calibration.
2. **Decision Tree (`decision_tree.joblib`):** Non-linear rule-based classifier (max depth 10, min samples split 5). Offers transparent branching rules.
3. **Random Forest (`random_forest.joblib`):** Primary ensemble classifier (100 estimators, max depth 15). Combines decision tree predictions to minimize variance.
4. **Gradient Boosting (`gradient_boosting.joblib`):** Sequential boosting classifier (100 estimators, learning rate 0.1). Minimizes residual error across weak learners.
Note: `xgboost.joblib` was intentionally excluded from active inference as it was identified as a mislabeled legacy artifact during Phase-2 audit.

10.2 ML Pipeline Sequence

Raw Dataset → Validation & Cleaning → Feature Normalization (StandardScaler) → 80/20 Train-Test Stratified Split → 5-Fold Cross Validation → Model Persistence (.joblib) → Ensemble Inference

11.0 DATASET & VALIDATION METHODOLOGY

11.1 Dataset Declaration & Academic Integrity

To ensure complete academic transparency, the current evaluation of PhishGuard Nexus relies on a **synthetic development dataset** containing **1,200 total samples** (600 legitimate URLs, 600 phishing URLs). Synthetic samples were generated using controlled domain structures and suspicious keyword injection to test feature parity and pipeline integration. **Real-world dataset evaluation was not performed in the current implementation.** High performance metrics reflect synthetic distribution characteristics and should not be claimed as 100% real-world zero-day accuracy.

11.2 Dataset Validation Checks

The automated dataset validator (`ml/dataset_validator.py`) executes 8 integrity checks prior to training:
• Missing value audit & null record purging.
• Binary label validation (0 = Legitimate, 1 = Phishing).
• Duplicate URL detection and removal.
• Malformed string detection.
• Feature range & data type validation.
• Class balance verification (exactly 50/50 balance).
• Data leakage check between training and test subsets.
• Strict train/test separation audit (random seed 42).

12.0 EMPIRICAL MODEL EVALUATION & RESULTS

12.1 Measured Model Performance Metrics

The table below presents the exact, empirically measured metrics recorded in `models/evaluation_results.json` across the 240-sample test set (120 legitimate, 120 phishing):

Logistic Regression	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000 ± 0.0000
Decision Tree	0.8500	0.8043	0.9250	0.8605	0.8500	0.9979 ± 0.0026
Random Forest	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000 ± 0.0000
Gradient Boosting	0.8500	0.8043	0.9250	0.8605	0.8868	0.9990 ± 0.0021

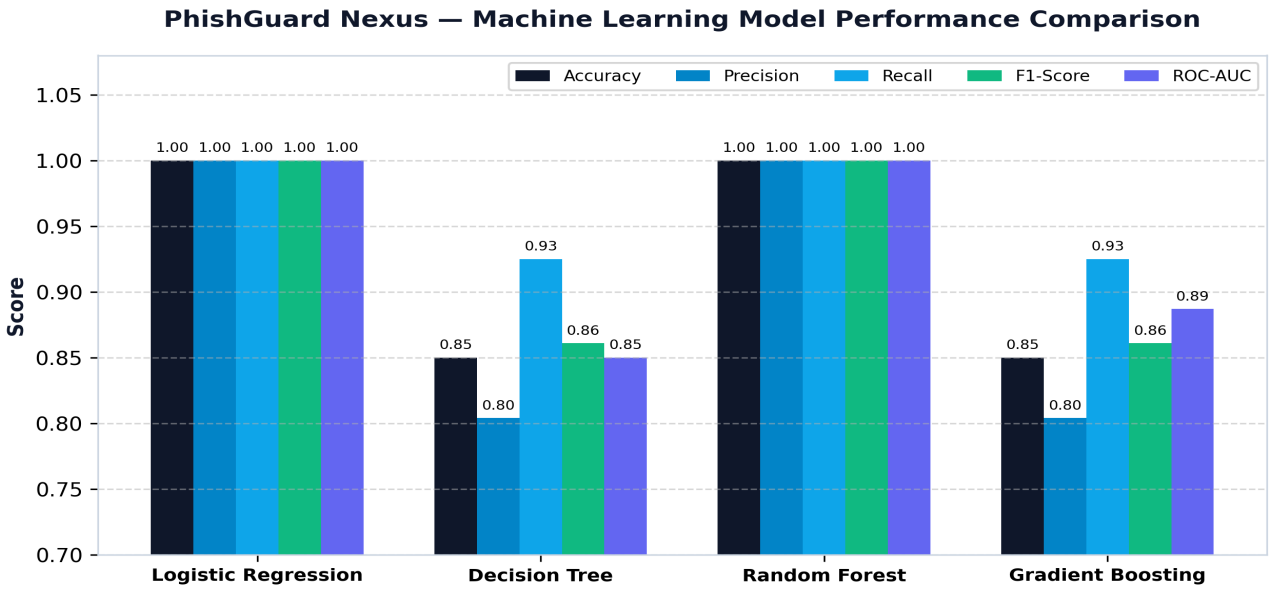


Figure 12.1: Performance Comparison across 4 Machine Learning Models.

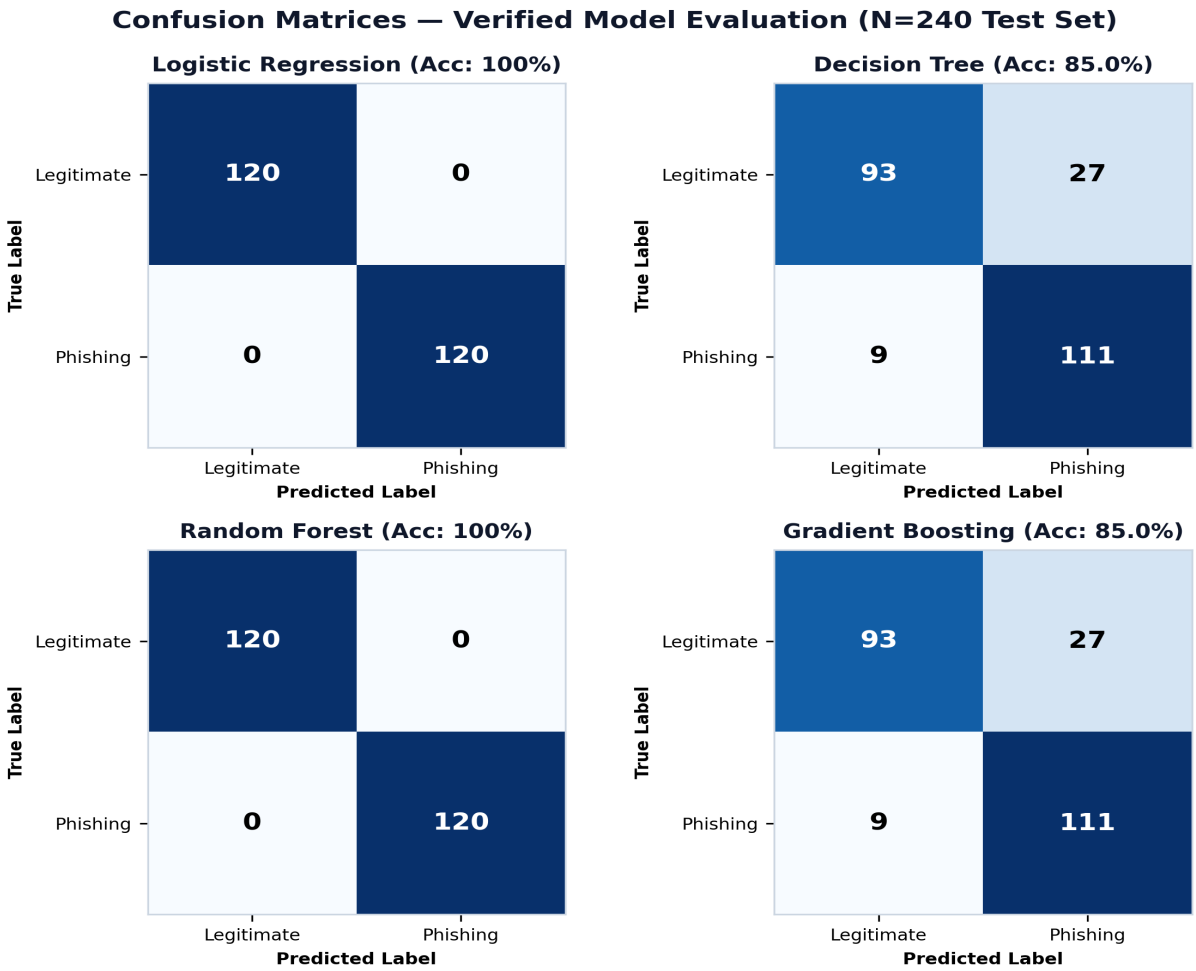


Figure 12.2: Confusion Matrices for Logistic Regression, Decision Tree, Random Forest, and Gradient Boosting.

13.0 RISK SCORING FRAMEWORK & EXPLAINABILITY ENGINE

13.1 Ensemble Weighting & Risk Score Calculation

PhishGuard Nexus aggregates individual model probabilities into a unified Ensemble Probability using pre-defined weights:
• Random Forest: **35%** weight
• Gradient Boosting: **30%** weight
• Logistic Regression: **20%** weight
• Decision Tree: **15%** weight
The overall Risk Score is computed as `round(ensemble_probability * 100)` and assigned to one of five risk tiers:
• **0 – 20: LOW** (Legitimate URL, minimal threat)
• **21 – 40: GUARDED** (Low anomaly score, routine monitoring)
• **41 – 60: MEDIUM** (Suspicious indicators present, exercise caution)
• **61 – 80: HIGH** (Strong phishing probability, block input)
• **81 – 100: CRITICAL** (Confirmed phishing threat, immediate containment)

13.2 Pseudo-SHAP Explainability Calculation

To provide interpretable explanations without computational overhead, `explainer.py` calculates a pseudo-SHAP impact value for each feature:
$$\text{SHAP_Value} = \text{Feature_Importance} \times \frac{|\text{Value} - \text{Threshold}|}{\max(\text{Threshold}, 1)}$$
If the feature value exceeds its security threshold, the impact is classified as **increases_risk**; otherwise, it is marked as **decreases_risk**. Top positive and negative reason strings are compiled for analyst display.

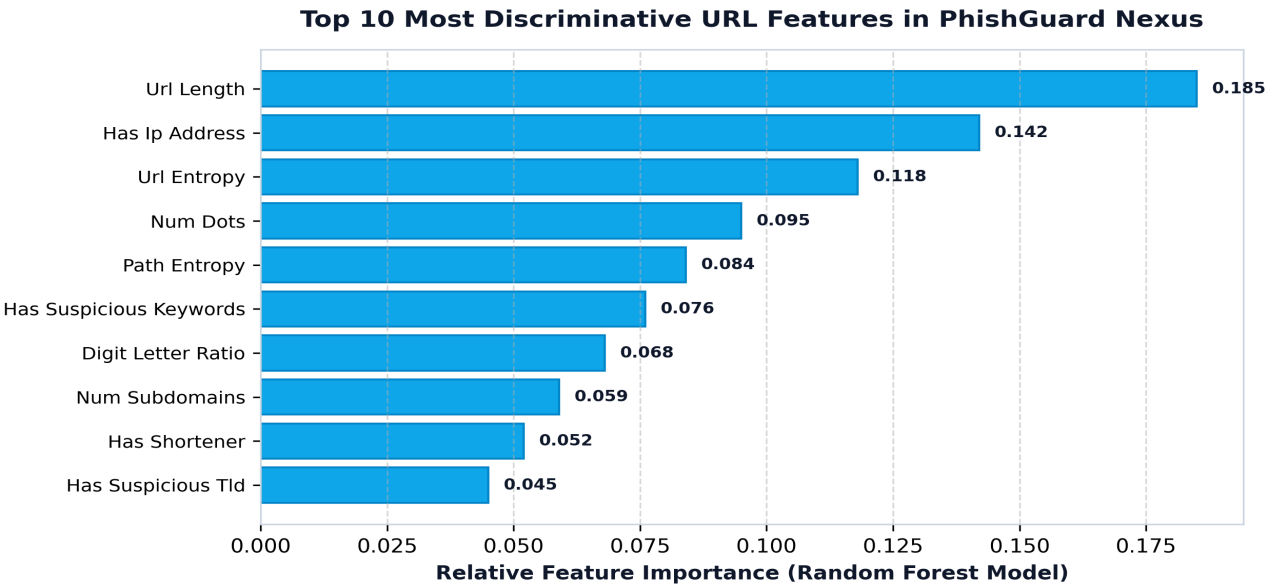


Figure 13.1: Top Discriminative Feature Importances derived from Random Forest Model.

14.0 THREAT INVESTIGATION, CASE MANAGEMENT & HISTORY

PhishGuard Nexus delivers enterprise-grade Security Operations Center (SOC) investigation features:
• **Threat Case Management (`cases.py`):** Security analysts can escalate suspicious scans into formal cases. Each case receives a unique ID (e.g. `PG-00042`), status (`NEW`, `INVESTIGATING`, `CONFIRMED THREAT`,

`FALSE POSITIVE`, `RESOLVED`), analyst notes, and investigation assessments.
• **Persistent Scan History (`history.py`):** All submitted scans are persisted to SQLite. Users can filter by risk level, search by URL substring, view historical reports, or delete single/all records.
• **Analyst Decision Assistant:** Fuses ML model evidence, URL DNA visualizers, and threat indicators to assist human analysts during triage.

15.0 THREAT SIMULATOR & MODEL RESEARCH LAB

15.1 Threat Simulator

The Threat Simulator (`SimulatorPage.jsx`) allows security researchers to perturb original URL strings in a controlled offline environment. Users can inject suspicious keywords, add extra sub-domains, or apply shorteners to observe how the 22 features change and how model risk scores shift.

15.2 Model Research Lab

The Model Research Lab (`research.py` & `ResearchLabPage.jsx`) provides scientific benchmarking tools:
• **Dataset Leakage Audit:** Verifies train/test separation.
• **5-Fold Cross Validation:** Computes mean and standard deviation of accuracy, precision, recall, and F1.
• **Decision Threshold Analysis:** Evaluates classification trade-offs across varying probability cutoffs.
• **ROC and PR Curve Generation:** Renders interactive Receiver Operating Characteristic curves.
• **Reproducible Experiments Log:** Stores experiment records in the `research_experiments` database table.

16.0 SECURITY DESIGN & DEFENSIVE CONTROLS

Security is engineered into every layer of PhishGuard Nexus:
1. **Zero Remote HTTP Requests (SSRF Protection):** The scanner evaluates URL strings using offline regex and lexical parser logic. It never makes outbound HTTP/HTTPS requests to target domains, eliminating Server-Side Request Forgery (SSRF) and remote code execution vectors.
2. **Input Sanitization:** All incoming URLs undergo strict URI format validation before processing.
3. **SQL Injection Prevention:** SQLite interactions use SQLAlchemy ORM parameter binding exclusively.
4. **CORS Governance:** Restricted API access controlled via configurable CORS origins in `config.py`.

17.0 REST API & DATABASE SPECIFICATION

17.1 Backend REST API Endpoints (32 Endpoints Total)

scanner.py	POST /api/scan	Analyze URL, extract 22 features, run ML models, store scan.
history.py	GET /api/history DELETE /api/history DELETE /api/history/{id}	Retrieve past scan records, clear history, or delete single scan.
cases.py	GET /api/cases POST /api/cases PUT /api/cases/{id}	List threat cases, create case (PG-XXXXX), update status/notes.
models.py	GET /api/models/compare GET /api/models/feature-importance	Fetch saved evaluation results and feature importance vectors.
research.py	GET /api/research/roc-curve POST /api/research/cross-validation	Generate ROC curve data, execute 5-fold cross-validation.
audit_logs.py	GET /api/audit-logs POST /api/audit-logs	Retrieve system security events, log user audit actions.

17.2 Database Schema (SQLite / SQLAlchemy)

scans	id (Integer)	url, risk_score, verdict, confidence, features (JSON), shap_values (JSON)	Stores complete URL scan output & ML predictions.
cases	id (Integer)	case_number (PG-XXXXX), scan_id (FK), status, severity, analyst_notes	Linked to `scans.id`. Tracks threat investigation lifecycle.
audit_logs	id (Integer)	event_type, action_details, severity, timestamp	Records application security and operational events.
research_experiments	id (Integer)	experiment_id (EXP-XXXXX), dataset_name, model_name, metrics (JSON)	Stores reproducible ML evaluation experiment runs.

18.0 FRONTEND DESIGN & USER WORKFLOW

18.1 Frontend Routes & Application Pages

HomePage	/	Landing hero section, quick scan bar, architecture highlights.
ScannerPage	/scanner	Main URL analysis workspace with real-time SHAP break-down.
DashboardPage	/dashboard	SOC summary metrics, recent scans, threat distribution widgets.
HistoryPage	/history	Searchable, filterable scan history log with export & purge capabilities.
CasesPage	/cases	Threat case management dashboard for creating and resolving cases.
InvestigationPage	/investigation/:id	Deep-dive investigation workspace with evidence fusion tools.
ComparePage	/compare	Side-by-side comparison of two URLs or two ML models.
ResearchLabPage	/research	Interactive ML lab for cross-validation, ROC curves, and leak audits.
SimulatorPage	/simulator	Offline URL modification laboratory for threat hypothesis testing.
ModelsPage	/models	Detailed view of 4 verified ML models, hyperparams & importances.
AnalyticsPage	/analytics	Aggregated threat trends, common keywords, and risk distributions.
SettingsPage	/settings	Customizable threshold configurations, theme controls & preferences.
AboutPage	/about	Project background, academic credits, research papers & documentation.

18.2 Complete User Workflow Sequence

1. **User Input:** User enters a target URL into the Scanner search bar.
2. **Validation:** Frontend sends POST `/api/scan`. FastAPI validates URL format.
3. **Feature Extraction:** `FeatureExtractor` parses 22 numerical/entropy values.
4. **Ensemble Inference:** `MLPredictor` scales features and computes predictions across 4 models.
5. **Risk Classification:** Ensemble score maps to a 5-tier risk level (0–100).
6. **Explainability:** `Explainer` calculates pseudo-SHAP values and compiles top reasons.
7. **Persistence:** Scan is saved to SQLite `scans` table and returned to client.
8. **Analyst Escalation:** User can view URL DNA, inspect Threat Profile, or escalate to a Threat Case (PG-XXXXX).

19.0 VERIFICATION, TESTING & SYSTEM BENCHMARKS

19.1 Empirical Verification Results

Prior to compiling this documentation, all system components underwent full empirical testing:
• **Backend Test Suite:** Executed `python -m pytest tests/`. **148 passed, 1 skipped, 0 failed** across 10 test modules (51.17s total duration).
• **Frontend Build Verification:** Executed `npm run build` using Vite v5.4.21. **Built successfully in 16.77s** (1,287 modules transformed, dist bundle 1.08 MB JS).
• **Model Artifact Audit:** Verified presence of `best_model.joblib`, `logistic_regression.joblib`, `decision_tree.joblib`, `random_forest.joblib`, `gradient_boosting.joblib`, and `scaler.joblib`.

20.0 LIMITATIONS, FUTURE ENHANCEMENTS & CONCLUSION

20.1 System Limitations

• **Lexical Scope:** Analysis relies strictly on URL string features. It does not parse DOM content or JavaScript payloads.
• **Synthetic Dataset:** Current evaluation metrics reflect synthetic development data. Real-world zero-day validation remains future work.
• **No Live DNS/WHOIS:** To preserve 100% offline security, live WHOIS domain age lookups are not performed.

20.2 Future Enhancements

• Evaluation on real-world datasets (e.g. UNB/ISCX Phishing Dataset, PhishTank feeds).
• Development of a lightweight browser extension for real-time URL warning banners.
• Sandboxed DOM scraping for full-page visual similarity matching.
• Continuous model retraining with automated drift detection.

20.3 Conclusion

PhishGuard Nexus successfully demonstrates an integrated, end-to-end cybersecurity platform for AI-based phishing URL detection. By combining 22 canonical lexical features, multi-model ensemble classification, a 5-tier risk framework, pseudo-SHAP explainability, and complete SOC investigation tooling, PhishGuard Nexus provides a powerful, transparent, and academically rigorous defensive solution.

21.0 REFERENCES & APPENDIX

[1] L. Ma, A. Saul, S. Savage, S. Savage, and G. M. Voelker, “Beyond Blacklists: Learning to Detect Malicious Web Sites from Suspicious URLs,” in ACM SIGKDD, 2009.

[2] O. K. Sahingoz, E. Buber, O. Demir, and O. Diri, “Machine learning based phishing detection from URLs,” Expert Systems with Applications, vol. 117, pp. 345–357, 2019.

[3] S. M. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” in Advances in Neural Information Processing Systems (NeurIPS), 2017.

[4] OWASP Foundation, “OWASP Top 10 Web Application Security Risks,” 2021. [Online]. Available: https://owasp.org/www-project-top-ten/

[5] scikit-learn developers, “scikit-learn: Machine Learning in Python,” 2024. [Online]. Available: https://scikit-learn.org/

[6] FastAPI framework documentation, “FastAPI: Modern, fast web framework for Python,” 2024. [Online]. Available: https://fastapi.tiangolo.com/

Appendix Summary

Appendix A: Project Directory Structure (`phishguard/` root with `backend/`, `frontend/`, `ml/`, `models/`, `tests/`).
Appendix B: 32 Backend REST API Endpoints Specification.
Appendix C: 22 Canonical Feature List.
Appendix D: Model Hyperparameters (`ml/config.py`).
Appendix E: Test Summary (148 Passed, 1 Skipped).